SILVER OAK UNIVERSITY

DEPARTMENT OF COMPUTER ENGINEERING

B.Tech. Computer Science and Engineering (Cyber Security)

MAJOR PROJECT – REVIEW 1 REPORT

Android Root Detection and SSL Pinning Assessment Platform: A Deterministic Detection, Verified Bypass and Calibrated Analysis Framework

Submitted by

Divyang Zinzuwadiya (2301030700073)

Heta Vaghela (2301030700068)

Harshita Rajput (2301030700046)

Project ID: CSE_CS_03

Guide: Mr. Shreyas Raybole

Head of Department: Dr. Vikas Tulshyan

Academic Year: 2026–27 | Review 1 | 24 October 2026

Abstract

Android applications routinely defend themselves with root detection and SSL/TLS certificate pinning, yet the tools commonly used to assess these defences stop at reporting that the mechanism is present. They do not demonstrate that the protection actually blocks an attacker, and they publish no measurement of how often their own verdicts are right. This report presents a deterministic analysis platform that closes that loop. Given an APK, the platform hashes and fingerprints it, records an authorization justification, scans five code surfaces against a declarative YAML signature corpus, classifies the application as ROOT_ONLY, SSL_ONLY, BOTH or NEITHER against a weighted-confidence threshold, and then executes only the dynamic bypass for the categories it actually detected. Each bypass is confirmed by a target-specific programmatic success check rather than by human observation. Every finding carries a verification state that distinguishes a rule match from an observed runtime behaviour, and every negative is certified only when the code surface that could hide the defence was fully scanned.

The platform is validated end-to-end against OWASP UnCrackable Levels 1, 2 and 3 and a wider calibration corpus of real store applications, 21 packages and 306 recorded runs in total. Against independently sourced ground truth it measures precision, recall and F1 of 1.00 on the subset that is fully covered and ground-truthed: 6 true positives, 1 true negative and no errors for root detection over 7 packages, and 2 true positives, 2 true negatives and no errors for SSL/TLS pinning over 4 packages. The dynamic confirmation rate is 1.00 in both categories among runs where verification actually executed, reported explicitly as a lower bound on precision rather than as precision itself. One measured false negative is disclosed rather than hidden. The contribution is therefore not a claim of perfect accuracy but a set of properties that make an accuracy claim auditable: cited evidence for every match, bounded negatives, calibrated confidence, and an immutable audit record binding each result to an operator, a justification and an exact code revision.

Keywords—*Android security, root detection, SSL/TLS certificate pinning, dynamic instrumentation, Frida, static analysis, calibration, OWASP MASTG.*

1. Introduction

Android holds the majority of the global mobile operating-system share, and a large proportion of the applications installed on it protect their own integrity at runtime. Two mechanisms account for most of that protection. Root detection inspects the device for signs of a compromised environment and refuses to run, or degrades its behaviour, when it believes it is being observed. SSL/TLS certificate pinning binds an application to a specific certificate or

public key, so that a certificate issued by any other authority — including one installed by an analyst for interception — is rejected. Together they form the first obstacle in almost every mobile assessment.

Assessing those defences is currently an artisanal process. An analyst decompiles the package, reads the decompiled Java and native code, forms an opinion about which checks are implemented, hand-writes a hook for the exact class and method names in that build, runs it on a rooted device, and then judges from the screen whether the bypass worked. Each of those steps depends on the individual, and none of them leaves a reproducible artifact behind. The resulting claims — that a protection was found, or that it was bypassed — are therefore assertions rather than measurements.

This project treats the problem as a measurement problem. It builds a platform that decides from explicit, readable rules, produces evidence that a reader can follow back to a file and a line number, verifies a bypass by observing the application's own check accept the instrumented environment, and records enough about each run to make the result reproducible and the accumulated accuracy measurable.

1.1 Scope of this report

This is a Review 1 report. It covers the problem statement, the motivation and relevance, the literature reviewed and the gap identified, the objectives, the tools and technology selected, the methodology, the results obtained so far and the conclusion. The deterministic core — ingestion, static detection, the decision engine, the dynamic bypass pipeline, correlation, reporting and calibration — is complete and validated. The three AI capability layers are designed and scoped but deliberately not built; Section 10 states the order in which they will be added and the gates they must pass.

2. Background and Motivation

The Android security model assumes the device is trusted. Once an attacker controls the operating system, every client-side check becomes advisory, because the attacker can modify the process that performs the check. Root detection and certificate pinning are therefore not cryptographic guarantees; they raise the cost of an attack by forcing the attacker to locate and defeat a specific piece of logic. That economic framing is what makes the two questions worth asking together: what does the application actually check, and does the check actually hold when someone tries to defeat it?

Industry practice answers the first question well and the second poorly. Static analysis frameworks such as MobSF and QARK decompile the package and report the presence of API calls associated with root checks and with pinning. That report is useful as a starting point, but

it conflates three different things: a protection the application implements itself, the same protection present inside a bundled third-party library, and a coincidental match on a string or a type name. A tool that cannot tell those apart produces findings that are not merely imprecise but structurally unverifiable.

Four specific failures recur across the tooling reviewed, and each one is a design decision this project makes differently.

- Detection without verification. A reported indicator is never executed against the application, so the tool cannot distinguish a functioning defence from dead code left in the build.
- No measurement of accuracy. No reviewed tool publishes precision or recall for its own verdicts, so an analyst has no basis for weighing a finding.
- Manual, non-reproducible bypass. The hook is written by hand against one build, and running the same tool tomorrow produces a different artifact with no link to the earlier evidence.
- Unbounded negatives. An absent detection is reported as a confident clean result even when the surface that could hide the defence — native libraries, a React Native bundle, an obfuscated payload — was never fully scanned.

The motivation follows directly. A platform that records what it scanned, cites what it matched, executes only the bypass that applies, confirms that bypass by observation, and keeps a running measurement of its own agreement with independent ground truth converts an analyst's opinion into a result that can be reviewed, re-run and challenged.

3. Relevance and Importance

The contribution is narrow and specific: the platform goes beyond detecting a protection to verifying that the protection can actually be bypassed, and it does so with programmatic confirmation instead of human observation. Because a bypass is confirmed by the application's own logic accepting the instrumented environment, the result is reproducible and carries a measured confidence rather than an impression.

The relevance is strongest where an unverified claim is most costly. Root detection and certificate pinning are implemented almost universally across banking, payment and fintech applications. In those applications the practical security question is not whether a pinning configuration file exists in the APK but whether a user with a modified device and an interception proxy is actually stopped. A finding that answers the first question and implies the second can lead an assessment in the wrong direction.

The work is relevant to four distinct audiences.

- Mobile security analysts and penetration testers, who gain a repeatable harness in place of a hand-written script per target.
- Researchers, who gain a corpus in which predicted confidence and independently established truth are stored side by side, making the detector's behaviour itself a subject of study.
- Students, who gain a worked demonstration that a measurement discipline is compatible with an offensive technique.
- Academic evaluators, who gain results that state their own exclusions next to the numbers, so the strength of a claim can be judged rather than trusted.

The methodology is aligned with the OWASP Mobile Application Security Testing Guide, which defines the static and dynamic test procedures this platform automates, and with the Mobile Application Security Verification Standard, which supplies the categories the platform classifies against. The architecture also provides a defensible foundation for later capabilities — knowledge-grounded reporting over the MASTG catalogue, semantic hypothesis generation over unmatched code, and human-reviewed bypass synthesis — because the evidence discipline that would keep those layers honest is already in place.

4. Literature Review

The literature relevant to this problem divides into four strands: methodology standards, static detection of Android security properties, runtime instrumentation and its adversarial side, and machine-learning approaches to code analysis. Table 1 summarises the works reviewed, in the format adopted for this review.

Table 1. Summary of the literature reviewed

Sr.	Ref.	Approach / algorithm	Dataset	Result	Findings
1	[1], [2]	OWASP MASTG and MASVS manual static and dynamic test cases	OWASP reference apps and CTF targets	Standardised procedure; no automated verification	Defines the practice but leaves execution and proof to the analyst
2	[3]	Androguard: static dissection and API call graph extraction	Arbitrary APK input	Programmatic access to manifest, DEX and resources	Supplies the parsing foundation a signature engine needs
3	[4]	DREBIN: static feature extraction with linear SVM classification	5,560 malware and 1,230 benign apps	94% detection accuracy	Static features scale well but yield no runtime proof
4	[5]	TaintDroid: system-level dynamic taint tracking	Real handset with instrumented runtime	High-precision taint propagation at runtime	Runtime instrumentation is reliable when the runtime is controlled
5	[6], [7]	Large-scale analysis of SSL usage in Android applications	13,500+ and 1,000+ store apps	Widespread TLS misconfiguration	Shows the scale of the problem, without bypass validation
6	[8]	SMV-HUNTER: static taint analysis with dynamic confirmation	Play Store application corpus	MITM vulnerabilities across many apps	Closest prior art; still not a reusable bypass harness
7	[9],	Anti-analysis and anti-	Samples with anti-	Evasion defeats	Dynamic findings must be

Sr.	Ref.	Approach / algorithm	Dataset	Result	Findings
	[10]	emulation evasion study	VM and anti-debug logic	naive sandboxes	verified, never assumed
8	[11]	Dynamic code loading analysis	Apps using DexClassLoader and reflection	Unsafe dynamic loading identified	Explains why static-only detection misses delegate payloads
9	[16]	FlowDroid: context-sensitive, lifecycle-aware taint analysis	Android application benchmarks	Precise flow tracking at high scalability cost	Precision costs runtime, motivating weighted signatures
10	[12], [13]	Deep learning vulnerability detection (VulDeePecker, Devign)	Synthetic and real C/C++ datasets	High recall, weak explainability	Justifies keeping the detector rule-based and explainable

Three observations follow from the review. First, the standards strand is mature and prescriptive; the gap is in executing and proving its tests automatically rather than in describing them. Second, static detection research demonstrates that rule-based and feature-based methods are accurate enough to be useful, and that explainability is what they trade for it — a trade this project accepts deliberately. Third, the dynamic strand establishes that instrumentation works and, in the same works, that adversaries actively detect and defeat it. That combination is precisely why a bypass must be observed rather than asserted.

5. Research Gap

Despite substantial work in each individual strand, the following gaps remain unaddressed in a single coherent system.

- Detection without verification. Reviewed tools report the presence of root checks or pinning but never execute the corresponding bypass to establish that the defence is effective.
- No accuracy measurement. No reviewed tool publishes precision, recall or a confidence-calibration curve for its own verdicts, so findings cannot be compared or weighted.
- Manual, non-reproducible bypass. Bypass work depends on an analyst hand-writing hooks per application, which produces results that vary between analysts and between runs.
- No separation of evidence types. A bundled library's own code is regularly read as the application's protection, because a pattern match and an observed behaviour are reported as the same kind of fact. The PropertyGuru case in Section 8.4 is a concrete instance found and corrected during this work.
- Unbounded negatives. An absent detection is reported as a confident negative even when the surface that could hide the defence was never fully scanned — the case for every application whose logic lives in a native library, an obfuscated payload or a JavaScript bundle.
- No accountable audit trail. Dual-use analysis tooling generally leaves no immutable record of who ran what, on which build, under what authorization and with which tool versions.

This project addresses each gap directly: automated bypass with a programmatic check, a two-tier calibration harness, a fixed and versioned pipeline, an explicit verification-state field, coverage-bounded negatives, and a mandatory justification with an append-only audit log.

6. Objectives

The overall aim is to design and implement a deterministic analysis platform that determines, for any APK, whether root detection, SSL/TLS certificate pinning, both or neither are present, and to verify any claimed bypass empirically. The aim decomposes into seven objectives.

O1. Build a declarative signature engine. A YAML-driven corpus matching across five surfaces — native libraries, Java bytecode, the Android manifest, resources and JavaScript bundles — producing file-and-line evidence for every match.

O2. Implement a weighted decision engine. Combine per-rule weights into a per-category confidence and classify against a configurable threshold as ROOT_ONLY, SSL_ONLY, BOTH or NEITHER.

O3. Automate a targeted dynamic bypass. Using ADB and Frida, provision the device, inject only the script for the detected category, and capture logcat throughout.

O4. Provide a programmatic success check per target. Confirm the bypass from the application's own behaviour rather than from a screenshot, and report the result as a boolean.

O5. Establish a calibration harness. Measure dynamic confirmation rate, and precision, recall, F1 and a confidence-calibration curve against independently sourced ground truth.

O6. Enforce authorization and audit. Require a justification for every run and write an immutable record binding the result to an operator, a package hash and the exact code revision.

O7. Keep AI strictly subordinate. Define verification states that prevent a model-proposed finding from ever being reported as a detected one, so later AI layers cannot acquire authority over a verdict.

7. Required Dataset, Tools and Technology

7.1 Programming languages and platforms

- Python 3.11 or later — the core engine, 60 source modules and approximately 31,000 lines of code.
- JavaScript — Frida instrumentation scripts, one per bypass technique and target.
- YAML — the declarative signature corpora and the ground-truth registry.
- SQL — result and run storage, with JSON retained as the audit-log format.

- WSL2 Ubuntu 24.04 (4 vCPU, 11 GB RAM) as the analysis environment, with a rooted Android 14 x86_64 AVD on the Windows host reached through adb reverse.

7.2 Toolchain

Table 2. Toolchain, pinned versions and role

Tool	Version	Role in the pipeline
jadx	1.5.1	APK to Java source decompilation; the corpus the bytecode signatures are matched against
apktool	3.0.1	Resource and smali extraction
aapt2	2.19	Manifest and metadata extraction
apksigner	0.9	Signing scheme verification and certificate digest
Frida	16.7.19	Runtime instrumentation, both server and client, for the bypass scripts
objection	1.11+	Assisted Frida exploration during technique development
ADB	platform-tools	Device control, install, reverse tunnelling, logcat capture
mitmproxy	8.1.1	Optional live traffic capture under the bypass, on the Windows host
Ghidra / capstone / pyelftools	current	Native library structure, symbols and disassembly
Typer / Rich	0.12+ / 13+	Command-line surface and console reporting
Jinja2	3.1+	Markdown and HTML report templates
SQLAlchemy / SQLite	2.0+	Run, APK and calibration storage
androguard / pyaxmlparser	4.1+ / 0.3+	Programmatic APK parsing
pytest	8.0+	150 unit test modules covering the deterministic core
uv	current	Dependency resolution and lockfile reproducibility

7.3 Dataset

The evaluation corpus comprises 21 packages in two groups. The first group is the OWASP UnCrackable series, Levels 1 to 3, which provide the primary end-to-end validation because their ground truth can be established by reading the decompiled source directly. The second group is 18 real applications drawn from public app stores, which supply the calibration breadth: they include AndroGoat, PropertyGuru, InsecureBankv2, Spotify, a certificate pinning demonstration build, and stock AOSP Contacts as a deterministic negative control. Ground truth is stored in config/ground_truth.yaml, keyed by APK SHA-256, and every entry is required to carry a citation recording how that truth was established independently of the pipeline's own predictions.

8. Methodology

The platform is implemented as a deterministic core of four phases with three AI capability layers deferred on top of it. The core is a single Python process exposing eight command-line commands; the pipeline itself runs as six ordered stages. Every stage returns a structured

status, and a stage that fails halts the run loudly rather than degrading into a guess — retries are reserved for tooling and environment failures, never for massaging an ambiguous verdict.

8.1 Stage 1 — Ingestion and authorization

The APK is read once and fingerprinted: SHA-256 digest, size, package name, version code, minimum and target SDK, signing schemes and certificate digest, all extracted with aapt2 and apksigner. Before any analysis begins, the run must supply a justification drawn from a closed set — OWN_APP, CTF, AUTHORIZED_ENGAGEMENT with an identifier, or ACADEMIC_TRAINING_TARGET. There is no default; the command refuses to run without one. An immutable audit record is then appended, containing the timestamp, the package hash, the justification, the operator identity, the tool versions and the exact git commit hash of the codebase at run time. That commit hash doubles as a reproducibility anchor.

8.2 Stage 2 — Static detection across five surfaces

Signatures are data, not code. Four YAML corpora — root detection, SSL pinning, native detection and packers — hold 112 rule entries in total, each with an identifier, a category, a weight, a pattern type (literal string, regular expression, or symbol) and one or more patterns. Adding a technique means editing YAML, not rewriting orchestration logic, and this was proven during the work: the same root signatures matched UnCrackable Level 2 and Level 3 with zero changes, because matching is content-based rather than tied to any class or package name.

Matching runs over five surfaces, and each contributes evidence independently:

- Java bytecode — the jadx-decompiled source tree, matched line by line so that every hit yields a file and a line number.
- Smali and resources — the apktool output, which survives cases where source decompilation is incomplete.
- Android manifest and resources — network security configuration, pin-set declarations, debuggable flags and exported components.
- Native libraries — ELF symbol tables, imported symbols and embedded strings via pyelftools and capstone.
- JavaScript bundles — Hermes and React Native payloads, matched against an idiom list, which is where several real applications keep their actual logic.

Per-surface coverage is recorded as complete, partial or unscannable. This is what later allows a negative to be certified or excluded rather than simply asserted.

8.3 Stage 3 — Weighted decision engine

Rule weights accumulate within each category, are clipped to the unit interval, and are compared against the configured threshold. Both categories use 0.70, the deliberately

uncalibrated starting point, and the threshold in force is written into the report so a later boundary change reads as a boundary change rather than as a rule change.

```
confidence(category) = min(1.0, Σ weight(matched rules))
if confidence(root) ≥ 0.70 and confidence(ssl) ≥ 0.70: BOTH
elif confidence(root) ≥ 0.70: ROOT_ONLY
elif confidence(ssl)  ≥ 0.70: SSL_ONLY
else: NEITHER  (certified only if every governing surface was fully scanned)
```

The engine also applies an attribution filter. A match inside a bundled library is not evidence about the application, so matches whose enclosing file belongs to vendor code are separated from application code before scoring. This rule exists because it was needed: an early signature for OkHttp's CertificatePinner matched OkHttp's own library source and scored a real production application at 0.90 for pinning that it did not implement as its own protection. The signature was revised to require the actual Builder construction call rather than the bare class name, and the TrustManager check was narrowed to require the class declaration to state that it implements the interface.

8.4 Stage 4 — Targeted dynamic bypass

The dynamic stage executes only what the decision engine selected. If root detection was detected, the root bypass runs. If pinning was detected, the SSL bypass runs. If neither was detected, the stage is skipped and the report says so with its reason, rather than running a bypass that has nothing to defeat. The sequence is:

- Step 1. Boot-check the rooted AVD and confirm it is reachable over ADB.
- Step 2. Install the APK, and auto-provision the version-matched frida-server binary if it is not already running.
- Step 3. Start logcat capture before the application launches.
- Step 4. Inject the selected Frida script, which hooks the specific check the static evidence identified.
- Step 5. Launch the application and drive it far enough to force the check to execute.
- Step 6. Evaluate the target's success check and record a boolean, together with the hook lines that fired.
- Step 7. Optionally, under --capture-traffic, run mitmproxy on the Windows host through adb reverse, and confirm that the application's own pin logic accepted a connection it would otherwise have rejected.

The success check is the part that distinguishes this stage from ordinary Frida usage. Each target supplies its own check: for UnCrackable Level 1 to 3 the check is satisfied when all of that level's hooks fire and the application's root logic returns false-positive-free; for an unknown target, a technology-level check confirms root bypass when the application's own check probes a root artifact and is answered not-rooted, and confirms SSL bypass when the application's own

pin check runs without rejecting a connection. A bypass is never reported as confirmed because a script loaded without an exception.

8.5 Stage 5 — Correlation and reporting

Static and dynamic results are compared and every disagreement is flagged explicitly as a correlation finding rather than silently resolved in favour of either side. The report is rendered twice, as Markdown and as HTML, from stored data, so a report can be regenerated later without re-running the analysis. Every report carries the analysis code hash, the signature-corpus fingerprint, the toolchain versions, the classification with its threshold, the per-category evidence list, the coverage and attribution limits, and an explicit limitations section.

8.6 Stage 6 — Calibration

Calibration is the mechanism that turns accumulated runs into an accuracy claim. It is computed by querying the run records directly, in a dependency-free module, and reports two tiers.

- Tier 1 — dynamic confirmation rate. Among runs where a category was predicted present and the dynamic stage actually executed, the fraction where a genuine bypass was confirmed. A not-confirmed run is never relabelled a false positive, because it is ambiguous between a false positive and a genuine detection whose bypass failed. The rate is therefore reported as a lower bound on precision, not as precision.
- Tier 2 — precision, recall and F1 against independent ground truth. Built from the static verdict alone, never from the bypass outcome, with one datapoint per package — that package's most recent run — because the detector's accuracy is a property of the application, not of how many times it was re-analysed. Every rate reports N/A rather than zero when its denominator is empty.

Two exclusion rules protect the honesty of the measurement. A negative is excluded when the surface that governs that category was partial or unscannable, because a negative from an incomplete scan is not proof of absence. And a package with no independent ground-truth entry for a category is excluded from that category entirely. Both exclusions are reported with their counts next to the result.

8.7 Evidence discipline

Every finding in the system carries a verification state drawn from a closed set: deterministic_matched, meaning the rule engine found it and the evidence is a file-and-line citation; dynamically_confirmed, meaning a real instrumentation run observed the predicted behaviour; ai_hypothesized_unverified, meaning a model proposed it and nothing has checked it; and ai_hypothesized_dynamically_confirmed, meaning a model proposed it and the harness

independently confirmed it. Nothing downstream — not the report, not the confidence score, not the calibration corpus, not the reusable technique library — treats the last two as equal to the first two. This single field is what allows the AI layers to be added later without ever letting a guess be presented as a fact.

9. Results and Discussion

9.1 End-to-end validation on UnCrackable Level 1 to 3

All three OWASP targets are handled by the identical pipeline, with no per-target orchestration logic and no new signature entries between them. Level 1 resolves to ROOT_ONLY with all three hooks firing and bypassed reported as true, promoting the finding from deterministic_matched to dynamically_confirmed. Level 2 resolves to ROOT_ONLY at 1.00 confidence, its root checks appearing byte-for-byte identical to Level 1 in a differently named class. Level 3 resolves to ROOT_ONLY at 1.00 confidence, its checks reappearing in a third class, together with a CRC32-based anti-tampering check that is deliberately excluded from both categories' confidence because it is neither root detection nor pinning.

Two original assumptions were falsified during this work and corrected rather than quietly dropped. Level 2 was expected to require native and JNI analysis; it did not, because its native library is unrelated to its root checks. Level 3 was expected to implement anti-Frida detection; a full-tree search of the decompiled source for tracerpid, ptrace, frida and port-scanning patterns returned nothing, so no such bypass was built. Both corrections are recorded in the handoff documents.

9.2 Tier 1 — dynamic confirmation rate

Table 3 reports the confirmation rate across the accumulated corpus. The counts are large because every bypass attempt is its own empirical observation; the rate describes only the runs in which verification actually executed.

Table 3. Tier 1 dynamic confirmation rate (lower bound on precision)

Category	Confirmed	Not confirmed	Unverified	N/A (not predicted)	Rate
Root detection	121	0	259	190	1.00
SSL/TLS pinning	10	0	202	358	1.00

The rate must be read with its denominator in mind. 259 root runs and 202 SSL runs were unverified: the dynamic stage did not execute, or the target's check was never reached. A thin count means not verified, not a zero per-cent success rate. This is why the figure is labelled a lower bound on precision and why Tier 2 exists.

9.3 Tier 2 — precision, recall and F1

Table 4 reports the measurement against independently sourced ground truth, computed from the static verdict alone. A perfect score on this subset is a statement about a small, fully covered and independently checked sample, not about the corpus as a whole.

Table 4. Tier 2 precision, recall and F1 against independent ground truth

Category	TP	FP	FN	TN	Precision	Recall	F1	n
Root detection	6	0	0	1	1.00	1.00	1.00	7
SSL/TLS pinning	2	0	0	2	1.00	1.00	1.00	4

The exclusions that define that subset are reported alongside it: 5 packages for root and 5 for SSL have no ground-truth entry, and 8 packages for root and 11 for SSL are excluded because their governing surface was partial, unscannable, or their negative was under-covered. Those exclusions are the honest cost of refusing to count an under-covered negative as a true negative. A detector willing to count them would report a larger and far less meaningful result.

9.4 Negative control and a disclosed false negative

Stock AOSP Contacts returns a clean confidence of 0.00 for both categories and a verdict of NEITHER. It is the deterministic true negative that establishes NEITHER as a reachable outcome rather than a theoretical one.

PropertyGuru, a real production application, returns root confidence 0.20 — below the threshold, and therefore a false negative against ground truth that says root detection is present. The cause is coverage, not mislabelling: the application's root checks live inside a React Native and Hermes bundle rather than in Java, so the Java signature corpus cannot attribute them. Because the negative comes from an under-covered surface it is excluded from the recall denominator, which means it does not depress the reported recall — and the calibration report says so in the same breath, naming the package and calling it a known miss. Reporting it is the point. A corpus that quietly omitted a known miss would report a recall it had not earned.

9.5 Scale of the accumulated evidence

Table 5. Corpus and implementation scale

Measure	Value
Packages in the corpus	21
Runs recorded	306
Reports generated	339
Linked audit records	983
Signature rule entries	112 across four YAML corpora
Unit test modules	150
Core source modules	60 (≈31,000 lines)

Three limits bound these figures. The corpus is small, so a single mis-entered ground-truth label would meaningfully move a rate — and one such mislabelling was in fact found and corrected during this work. Ground truth is maintained by hand, so the measurement rests on manual data. And the confirmation rate depends on instrumentation succeeding against each defence, so its value of 1.00 describes the runs where verification executed and not the corpus as a whole.

10. Conclusion

This work treats Android root detection and certificate pinning as measurement problems rather than pattern-matching problems. The resulting platform delivers five properties together: detection carrying cited evidence, bypass verified at runtime rather than by inspection, negatives bounded by measured coverage, confidence calibrated from real outcomes, and every run authorized and auditable.

Validated across OWASP UnCrackable Levels 1 to 3 and 18 further real applications over 306 runs, it measures precision, recall and F1 of 1.00 on the subset that is fully covered and independently ground-truthed, and it reports the exclusions that define that subset next to the number. The honest properties are the contribution: a finding always states how it was obtained, a negative is either certified under complete coverage or excluded, and no model can override a deterministic verdict. The deterministic core is delivered as working software, with the AI capability layers scoped and gated rather than promised.

11. Future Work

The remaining work follows a deliberate order: each layer is added only after the core is proven, and each is gated on empirical verification before anything it produces is trusted.

F1. Knowledge-grounded reporting. Index the OWASP MASTG technique catalogue and prior findings into a local vector store so that reports cite the actual technique rather than letting a model invent terminology, and so that structural version diffs between two builds of an application are narrated from retrieved context.

F2. Semantic hypothesis generation. For code the deterministic engine cannot confidently match, a heuristic pre-filter narrows candidates by suspicious API references, complexity near network and crypto code, and unmatched neighbours of matched code, before a code-reasoning model proposes structured hook targets. Proposals are stored as ai_hypothesized_unverified until the dynamic harness checks them, and the model is admitted only after a blind evaluation in which it must independently rediscover the known patterns.

F3. Human-reviewed bypass synthesis. A generate-and-verify loop in which a model drafts candidate hooks constrained to a small library of safe shapes, executed in a reverted emulator snapshot with a timeout and a bounded retry count. Promotion to a trusted, reusable technique requires two independent positive signals and explicit human review.

F4. Wider native coverage. Extend structural and string analysis for symbol-stripped native pin logic, where indicators are currently found but not attributed to a function.

F5. A larger independent corpus. Since the size of the measured subset is what limits the strength of any precision claim, growing the ground-truthed corpus is the highest-value deterministic next step.

F6. Full positive and negative screenshot verification. Add an independent vision signal during dynamic runs, alongside logcat parsing, so that on-screen state is confirmed by two modalities rather than one.

One thing will not change. Hardware-backed remote attestation, such as full Play Integrity verification, is documented as a permanent limitation rather than approached. That problem is verified server-side, and client-side hooking cannot honestly defeat it; claiming otherwise would be the exact failure this project exists to avoid.

12. References

[1] OWASP Foundation, "OWASP Mobile Security Testing Guide," 2024. [Online]. Available: https://owasp.org/www-project-mobile-security-testing-guide/

[2] OWASP Foundation, "Mobile Application Security Verification Standard (MASVS)," version 2.1, 2024. [Online]. Available: https://mas.owasp.org/MASVS/

[3] A. Desnos, "Androguard: Reverse engineering, malware and goodware analysis of Android applications," 2011. [Online]. Available: https://github.com/androguard/androguard

[4] D. Arp, M. Spreitzenbarth, M. Hubner, H. Gascon, and K. Rieck, "DREBIN: Effective and explainable detection of Android malware in your pocket," in Proc. Network and Distributed System Security Symposium (NDSS), 2014.

[5] W. Enck, P. Gilbert, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, "TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones," in Proc. 9th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2010.

[6] S. Fahl, M. Harbach, T. Muders, L. Baumgartner, B. Freisleben, and M. Smith, "Why Eve and Mallory love Android: An analysis of Android SSL (in)security," in Proc. ACM Conf. Computer and Communications Security (CCS), 2012.

[7] S. Fahl, M. Harbach, H. Perl, M. Koetter, and M. Smith, "Rethinking SSL development in an appified world," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2013.

[8] D. Sounthiraraj, J. Sahs, G. Greenwood, Z. Lin, and L. Khan, "SMV-HUNTER: Large scale, automated detection of SSL/TLS man-in-the-middle vulnerabilities in Android apps," in Proc. NDSS, 2014.

[9] T. Petsas, G. Voyatzis, E. Athanasopoulos, M. Polychronakis, and S. Ioannidis, "Rage against the virtual machine: Hindering dynamic analysis of Android malware," in Proc. 7th European Workshop on System Security (EuroSec), 2014.

[10] S. Rasthofer, S. Arzt, M. Miltenberger, and E. Bodden, "Harvesting runtime values in Android applications that feature anti-analysis techniques," in Proc. NDSS, 2016.

[11] S. Poeplau, Y. Fratantonio, A. Bianchi, C. Kruegel, and G. Vigna, "Execute this! Analyzing unsafe and malicious dynamic code loading in Android applications," in Proc. NDSS, 2014.

[12] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A deep learning-based system for vulnerability detection," in Proc. NDSS, 2018.

[13] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in Proc. NeurIPS, 2019.

[14] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[15] K. W. Y. Au, Y. F. Zhou, Z. Huang, and D. Lie, "PScout: Analyzing the Android permission specification," in Proc. ACM CCS, 2012.

[16] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Octeau, and P. McDaniel, "FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps," in Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI), 2014.

[17] Frida — dynamic instrumentation toolkit for Android, iOS and native apps. [Online]. Available: https://frida.re

[18] Mobile Security Framework (MobSF). [Online]. Available: https://github.com/MobSF/Mobile-Security-Framework-MobSF

[19] jadx — Dex to Java decompiler. [Online]. Available: https://github.com/skylot/jadx

[20] Google, "Android Network Security Configuration: certificate pinning," Android Developers. [Online]. Available: https://developer.android.com/privacy-and-security/security-config